

Rising Waters: A Machine Learning Approach to Flood Prediction

Project Description:

Rising Waters is a machine learning-based flood prediction system that predicts flood risk using historical weather and rainfall parameters. The web application is built using Flask and a trained Random Forest model. Users enter temperature, humidity, cloud cover, annual rainfall, seasonal rainfall, average June rainfall, and sub-division rainfall values to obtain a flood risk prediction and probability. The system helps authorities and citizens make timely decisions for disaster preparedness.

Scenarios:

Scenario 1: Early Flood Warning for Disaster Management

During the monsoon season, the Disaster Management Department collects weather parameters such as temperature, humidity, cloud cover, annual rainfall, and seasonal rainfall. These values are entered into the Rising Waters application. The machine learning model analyses the data and predicts whether there is a High Flood Risk or Low Flood Risk, along with the probability. If the flood probability is high, authorities can issue early warnings, evacuate people from vulnerable areas, and prepare emergency response teams to reduce the impact of flooding.

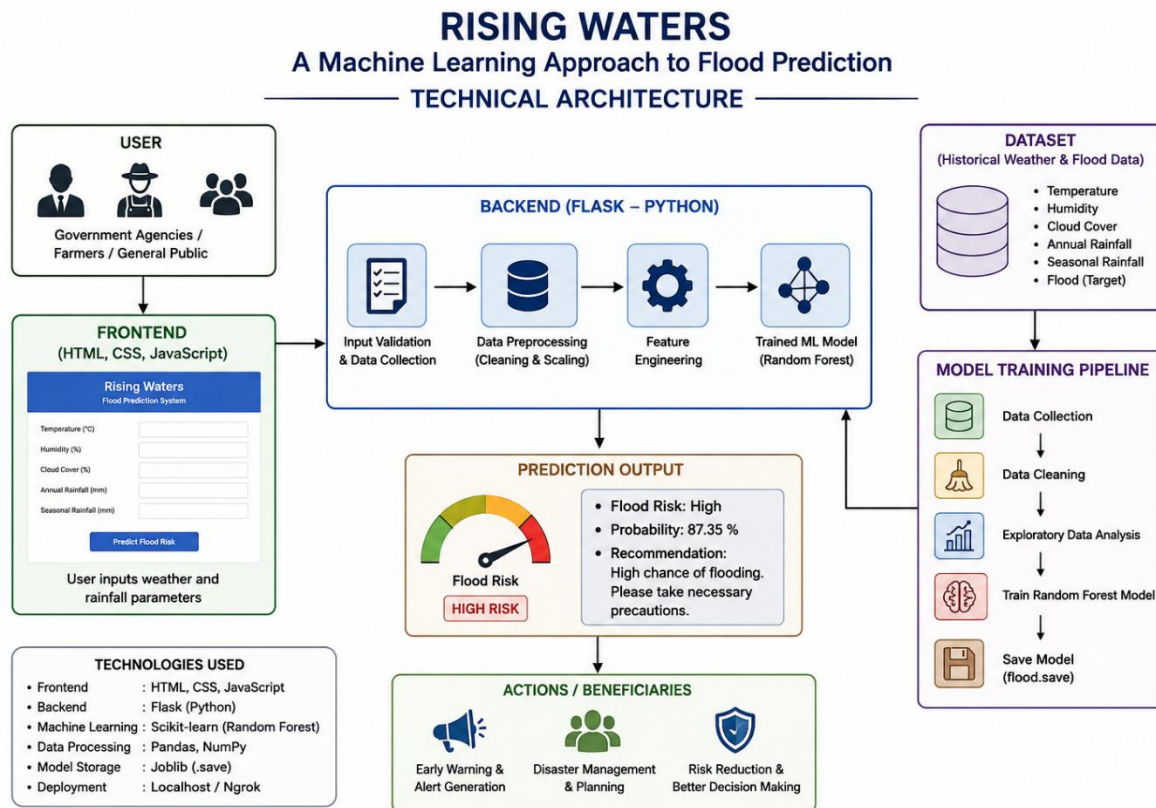
Scenario 2: Flood Risk Assessment for Farmers

A farmer wants to know whether the upcoming weather conditions may cause flooding in agricultural fields. The farmer enters the current weather and rainfall information into the application. The system predicts the likelihood of flooding and displays the flood risk. Based on the prediction, the farmer can postpone sowing, protect crops, move agricultural equipment to safer locations, and take preventive measures to minimize crop losses.

Scenario 3: Infrastructure and Urban Planning

Municipal authorities monitor weather conditions before periods of heavy rainfall. Using the Rising Waters application, they analyze the flood risk for a particular region by entering the required environmental parameters. If the application predicts a high flood probability, the authorities can clean drainage systems, deploy pumps, inspect reservoirs, and prepare emergency services. This helps reduce urban flooding, protect public infrastructure, and improve community safety.

Technical Architecture:



Description: The **Rising Waters – Flood Prediction System** uses a web-based architecture integrated with a machine learning model. Users enter weather and rainfall details through the web interface, and the data is sent to the Flask backend. The backend validates and preprocesses the input before passing it to the trained **Random Forest** model. The model predicts the flood risk and probability, and the results are displayed on the webpage along with safety recommendations. This architecture enables fast, accurate, and real-time flood risk prediction.

Database: Not used. The system uses a pre-trained Random Forest model stored with Joblib. User inputs are processed temporarily in the Flask backend to generate flood predictions and are not stored permanently.

Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Flask Installation Guide](#)

- **Machine Learning Concepts:** Knowledge of supervised machine learning, particularly the **Random Forest Classifier**, along with data preprocessing and feature scaling techniques
- **Dataset Handling:** Understanding of CSV datasets, data cleaning, preprocessing, feature selection, and model training.

Project Workflow:

Milestone 1: Dataset Collection and Preprocessing and perform EDA

- **Activity 1.1:** Collect the historical flood and rainfall dataset from reliable sources.
- **Activity 1.2:** Exploratory Data Analysis (EDA)
- **Activity 1.2:** Clean the dataset by handling missing values and selecting relevant features.
- **Activity 1.3:** Perform data preprocessing and feature scaling for model training.

Milestone 2: Machine Learning Model Development

- **Activity 2.1:** Train the Random Forest Classifier using the preprocessed dataset.
- **Activity 2.2:** Evaluate the model using performance metrics such as accuracy and prediction results.
- **Activity 2.3:** Save the trained model and scaler using Joblib for deployment.

Milestone 3: Frontend Development

- **Activity 3.1:** Design and develop a responsive user interface using HTML, CSS, and JavaScript.
- **Activity 3.2:** Create input forms to collect weather and rainfall parameters from users.
- **Activity 3.3:** Design result pages to display flood risk, probability, and safety recommendations.

Milestone 4: Backend Development

- **Activity 4.1:** Develop the Flask backend to handle user requests and routing.
- **Activity 4.2:** Load the trained machine learning model and preprocess user inputs.
- **Activity 4.3:** Generate flood predictions and return results to the frontend.

Milestone 5: Integration, Testing, and Deployment

- **Activity 5.1:** Integrate the frontend with the Flask backend for real-time prediction.
- **Activity 5.2:** Test the application using different weather and rainfall inputs to verify prediction accuracy.
- **Activity 5.3:** Deploy the application locally and manage the project using Git and GitHub.

Milestone 6: Documentation and Project Demonstration

- **Activity 6.1:** Prepare the project documentation, including architecture, workflow, and user guide.
- **Activity 6.2:** Create screenshots and a demonstration video showcasing the application's features.
- **Activity 6.3:** Finalize the project repository with the source code, README, and supporting documents for submission.

Milestone 1: Dataset Collection and preprocessing

This milestone focuses on preparing high-quality data for training the machine learning model. Historical flood, rainfall, and weather datasets are collected from reliable sources. The dataset is then cleaned by removing missing values, duplicate records, and irrelevant information to improve data quality. Important features such as **Temperature, Humidity, Cloud Cover, Annual Rainfall, Jan-Feb Rainfall, Mar-May Rainfall, Jun-Sep Rainfall, Oct-Dec Rainfall, Average June Rainfall, and Subdivision Rainfall** are selected for analysis. Finally, the data is preprocessed by applying feature scaling and formatting techniques to ensure consistency and improve the performance of the Random Forest model during training.

Activity 1.1: Collect Historical Flood and Weather Dataset

The first step involves collecting historical flood and weather data from reliable sources. The dataset includes important parameters such as Temperature, Humidity, Cloud Cover, Annual Rainfall, Jan-Feb Rainfall, Mar-May Rainfall, Jun-Sep Rainfall, Oct-Dec Rainfall, Average June Rainfall, and Subdivision Rainfall, along with the target variable Flood (Yes/No). This dataset serves as the foundation for training the machine learning model.

For this Flood Prediction, we take dataset from Kaggle that is flood_dataset.xlsx

Activity 1.2: Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is performed to understand the characteristics of the dataset. Statistical summaries and visualizations are used to identify data distributions, correlations between features, class imbalance, missing values, and outliers. Graphs such as histograms, box plots, correlation heatmaps, and count plots help reveal patterns in the data and provide insights into the factors that influence flood occurrence. These insights guide the preprocessing and model development stages.

Here, describe() function is used to find all required measures. They are Count, mean, std, min, max, 25%, 50%, 75% etc..

jupyter Flood_prediction Last Checkpoint: yesterday

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```

Jun-Sep      0
Oct-Dec      0
avgjune      0
sub          0
flood        0
dtype: int64

[8]: df.duplicated().sum()

[8]: np.int64(0)

[9]: df.describe()

[9]:
```

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
count	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000
mean	29.600000	73.852174	36.286957	2925.487826	27.739130	377.253913	2022.840870	497.636522	218.100870	439.801739	0.139130
std	1.122341	2.947623	4.330158	422.112193	22.361032	151.091850	386.254397	129.860643	62.547597	210.438813	0.347597
min	28.000000	70.000000	30.000000	2068.800000	0.300000	89.900000	1104.300000	166.600000	65.600000	34.200000	0.000000
25%	29.000000	71.000000	32.500000	2627.900000	10.250000	276.750000	1768.850000	407.450000	179.666667	295.000000	0.000000
50%	30.000000	74.000000	36.000000	2937.500000	20.500000	342.000000	1948.700000	501.500000	211.033333	430.600000	0.000000
75%	31.000000	76.000000	40.000000	3164.100000	41.600000	442.300000	2242.900000	584.550000	263.833333	577.650000	0.000000
max	31.000000	79.000000	44.000000	4257.800000	98.100000	915.200000	3451.300000	823.300000	366.066667	982.700000	1.000000

```

[10]: print("Shape of Dataset:", df.shape)

Shape of Dataset: (115, 11)

[11]: print("Duplicate Rows:", df.duplicated().sum())

Duplicate Rows: 0

```

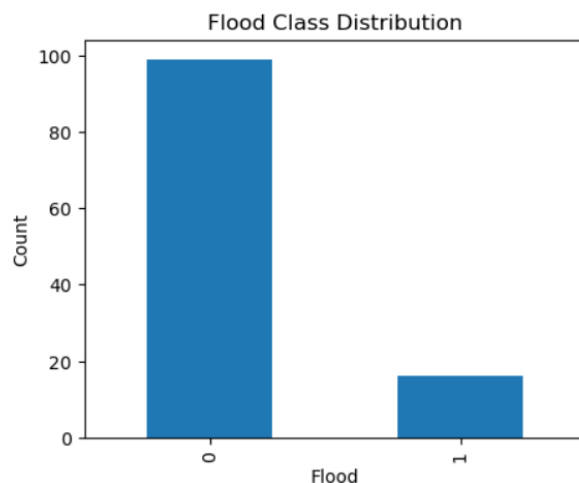
Flood Class Distribution:

```

[12]: import matplotlib.pyplot as plt

plt.figure(figsize=(5,4))
df['flood'].value_counts().plot(kind='bar')
plt.title("Flood Class Distribution")
plt.xlabel("Flood")
plt.ylabel("Count")
plt.show()

```



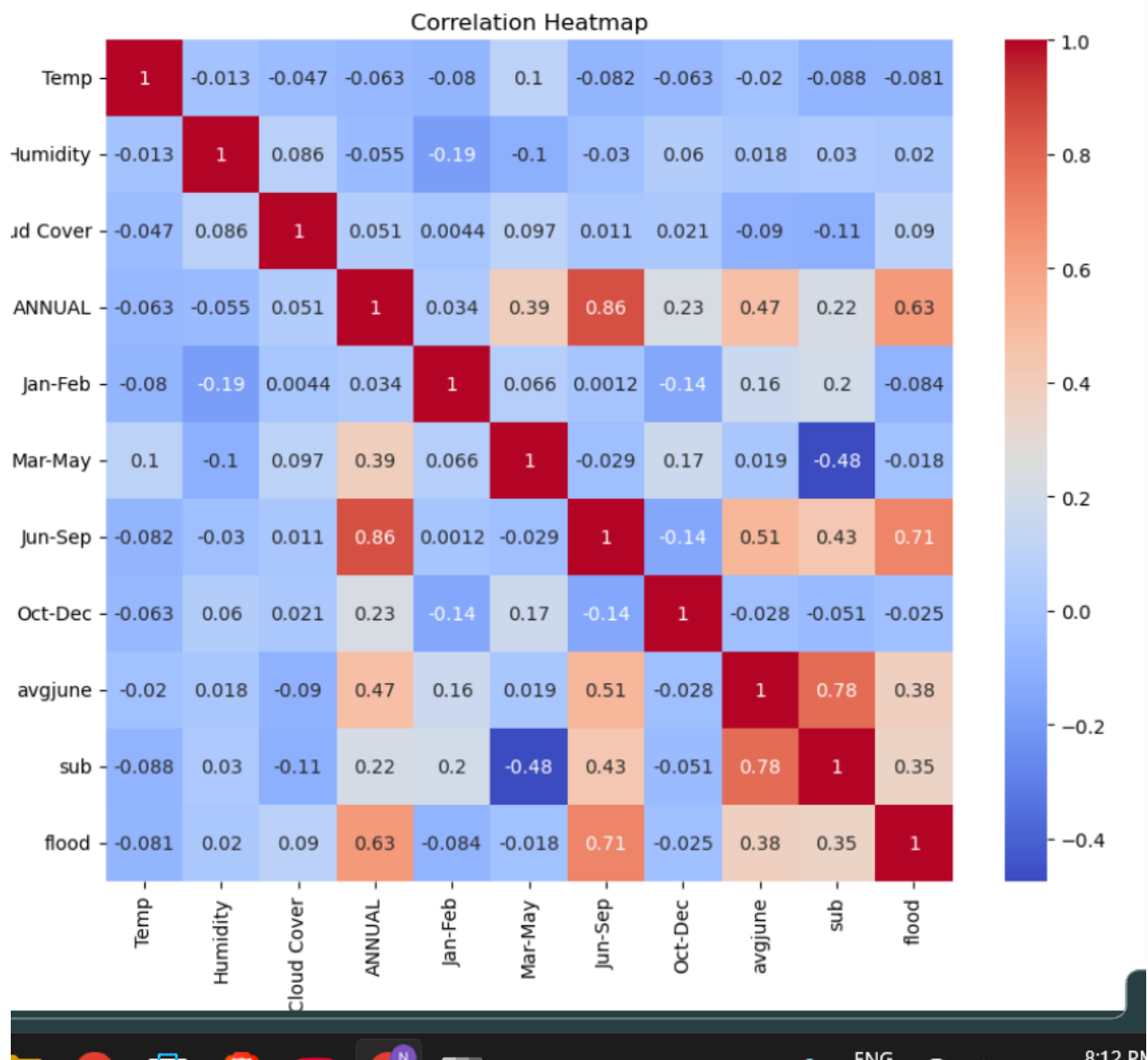
```

[13]: import seaborn as sns

```

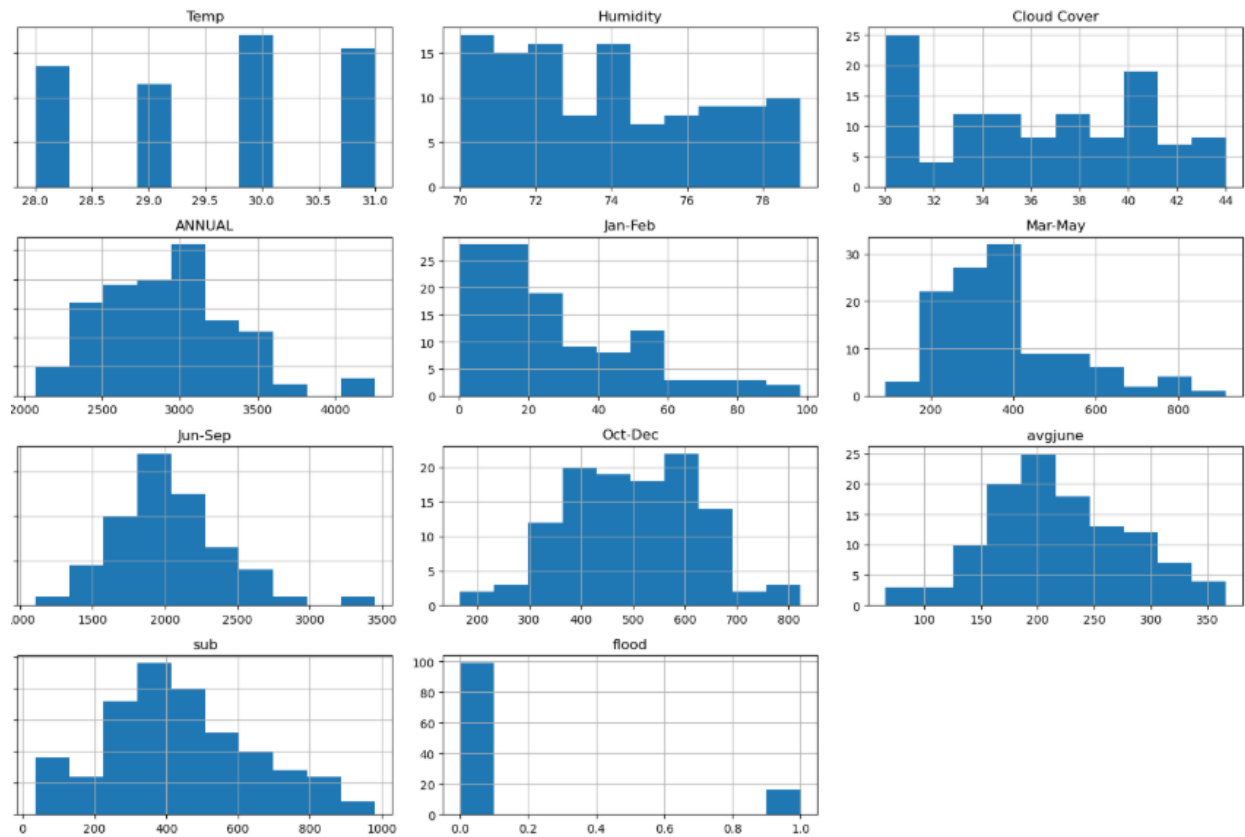
Correlation Heatmap:

```
l.title('Correlation heatmap')
l.show()
```



Histogram:

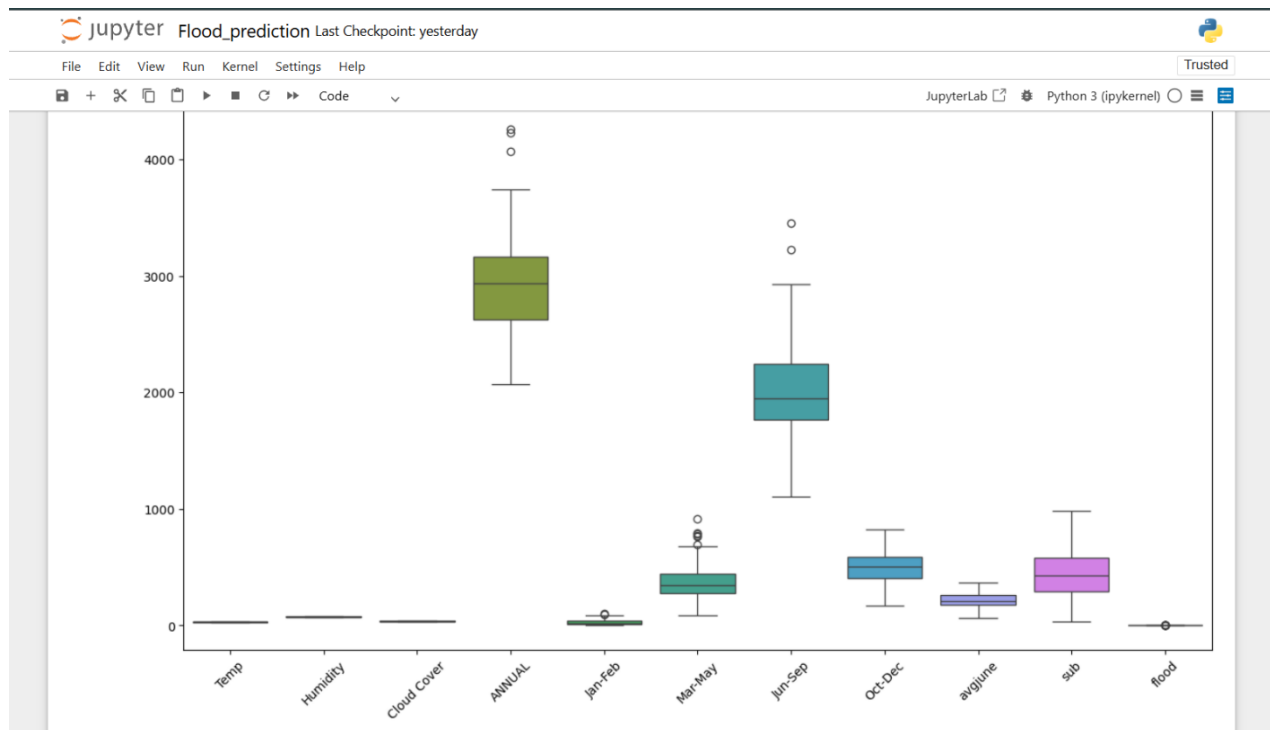
```
plt.tight_layout()
plt.show()
```



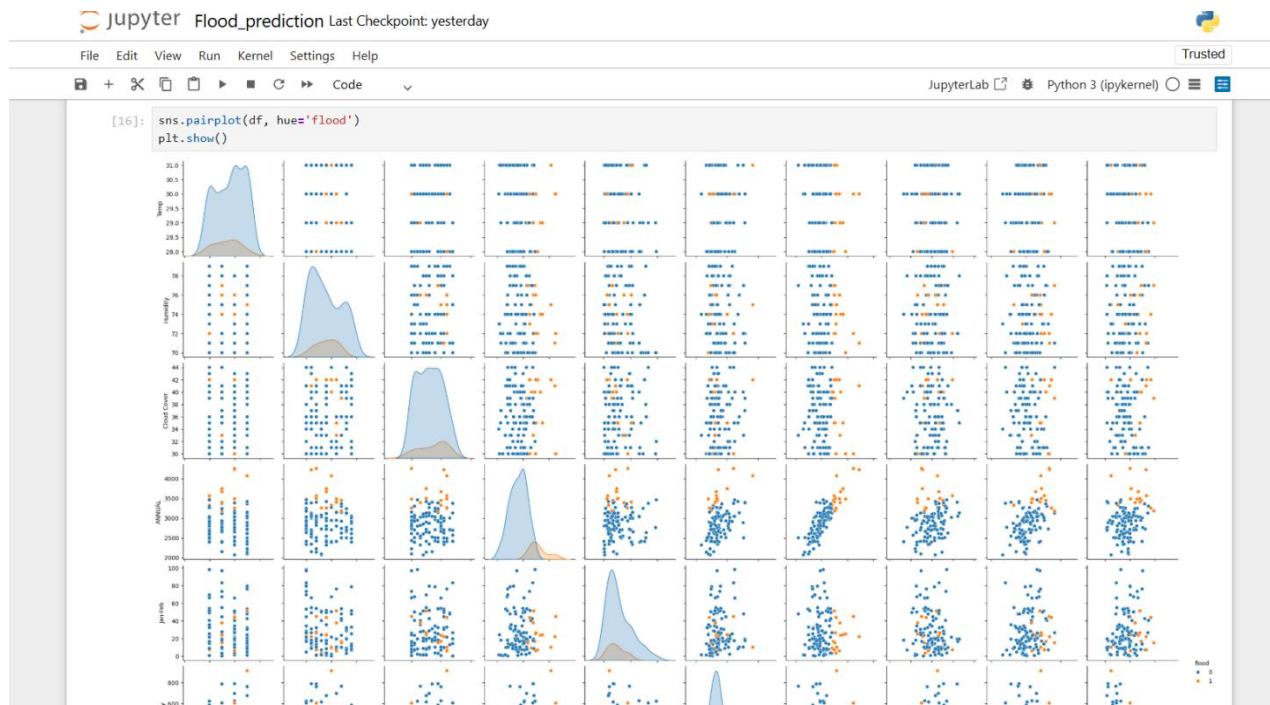
```
21]:
```

```
plt.figure(figsize=(15,8))
sns.boxplot(data=df)
```

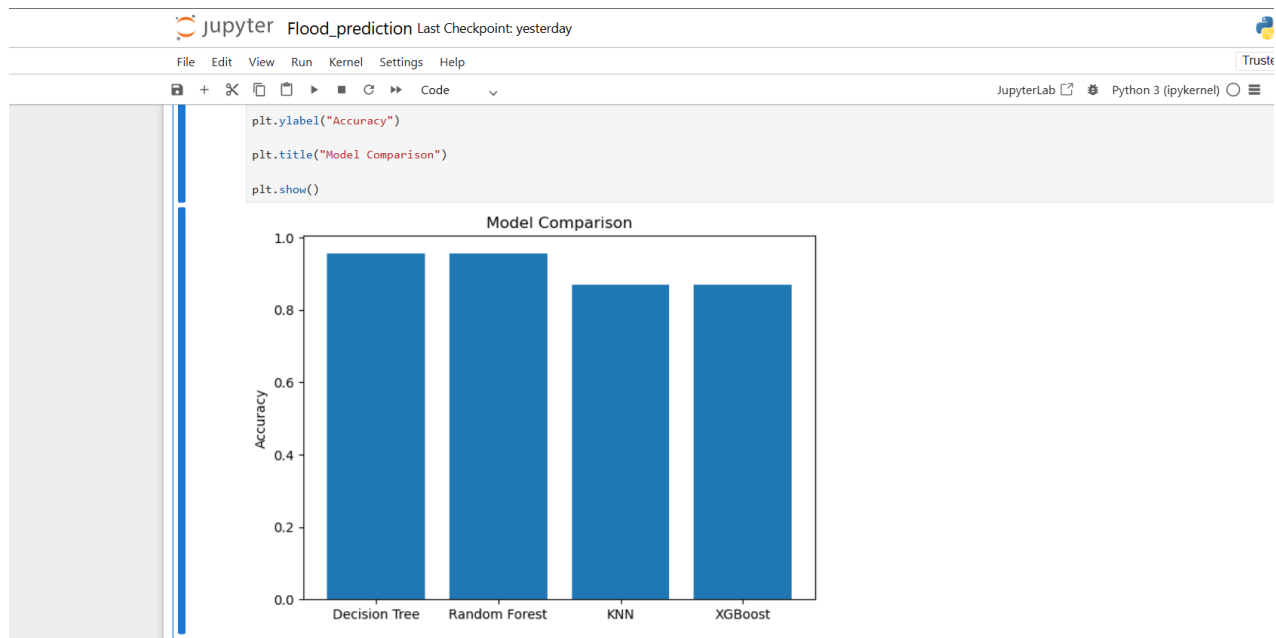
Boxplots:



Pairplot:



Model Comparison:



The Above diagram is **Decision Tree Confusion Matrix**

Like above, it has been explore the data using Exploratory Data Analysis and using different plots, graphs and comparison, it has finally got a best model to training and Prediction

Activity 1.3: Data Cleaning and Feature Selection

The dataset is cleaned by handling missing values, removing duplicate records, and correcting inconsistent data. After cleaning, the most relevant features affecting flood prediction are selected for model training. This step improves data quality and enhances the prediction performance of the machine learning model.

Activity 1.4: Data Preprocessing and Feature Scaling

The cleaned dataset is preprocessed by converting it into a suitable format for machine learning. Numerical features are standardized using StandardScaler to ensure that all variables are on a similar scale. Finally, the dataset is divided into training and testing sets, preparing it for training and evaluating the Random Forest Classifier.

Milestone 2: Machine Learning Model Development

This milestone focuses on developing a robust machine learning model for flood prediction. After completing data collection, EDA, and preprocessing, the prepared dataset is used to train the Random Forest Classifier. The trained model is then evaluated using various performance metrics to measure its accuracy and reliability. Finally, the model and preprocessing scaler are saved for integration into the Flask web application, enabling real-time flood risk prediction.

Activity 2.1: Model Selection and Training

In this activity, the Random Forest Classifier is selected as the machine learning algorithm for flood prediction because of its high accuracy, robustness, and ability to handle complex datasets. The preprocessed training dataset is used to train the model so that it can learn the relationship between weather parameters and flood occurrences.

```

jupyter Flood_prediction Last Checkpoint: yesterday
File Edit View Run Kernel Settings Help
+ ✂ 📄 📄 ▶ ■ ↺ ⏩ Code ▾ Jupyterl

joblib.dump(scaler, "scaler.save")

print("Model saved successfully!")

Model saved successfully!

[55]: probabilities = xgb_model.predict_proba(X_test)

print(probabilities[:5])

[[0.9875717 0.0124283]
 [0.9875717 0.0124283]
 [0.9875717 0.0124283]
 [0.9875717 0.0124283]
 [0.9875717 0.0124283]]

[56]: import joblib

joblib.dump(rf_model, "floods.save")
joblib.dump(scaler, "scaler.save")

print("Model and scaler saved successfully!")

Model and scaler saved successfully!

[57]: model = joblib.load("floods.save")
print(type(model))
print(hasattr(model, "predict_proba"))

<class 'sklearn.ensemble._forest.RandomForestClassifier'>
True

```

Activity 2.2: Model Evaluation and Performance Analysis

After training, the model is evaluated using the testing dataset to measure its prediction performance. Evaluation metrics such as **accuracy**, **confusion matrix**, **precision**, **recall**, and **F1-score** are analyzed to determine the effectiveness of the model. The prediction results are compared with the actual flood status to ensure reliable and accurate flood risk prediction.

Activity 2.3: Model Saving and Deployment Preparation

Once the model achieves satisfactory performance, it is saved using the Joblib library. Along with the trained Random Forest model, the StandardScaler used during preprocessing is also saved. These saved files are later loaded into the Flask application to provide real-time flood risk predictions without retraining the model every time the application is executed.

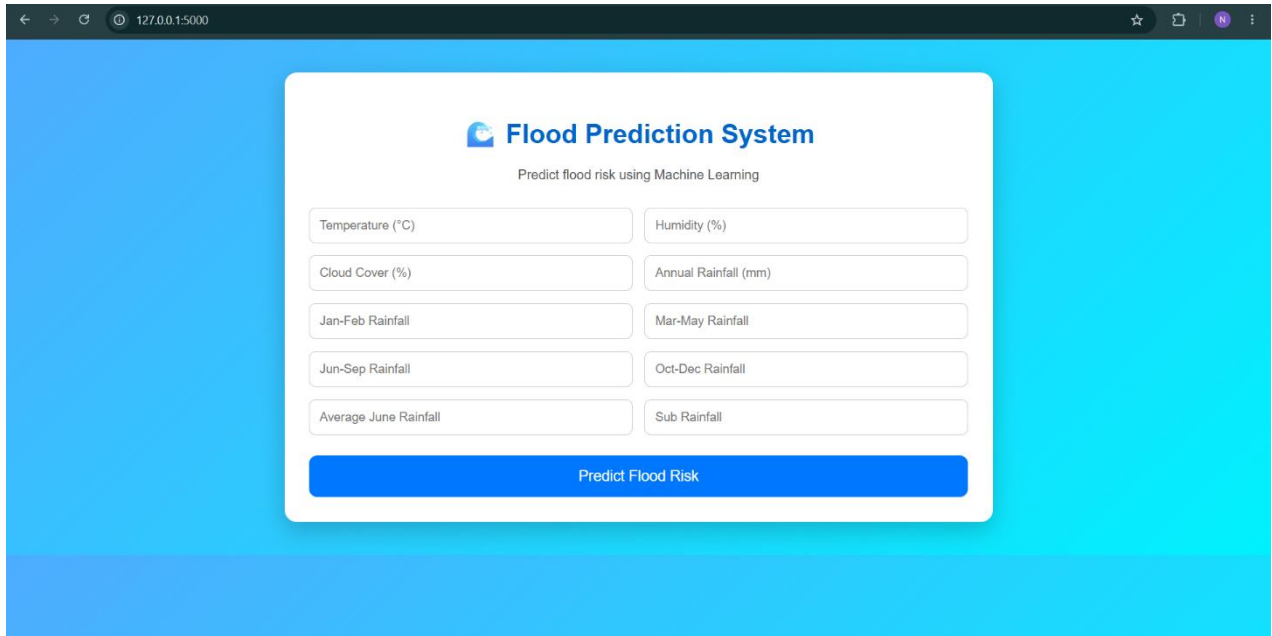
Milestone 3: Frontend Development

This milestone focuses on designing and developing a user-friendly web interface for the **Rising Waters – Flood Prediction System**. The frontend is built using **HTML**, **CSS**, and **JavaScript** to provide an interactive and responsive experience. It allows users to enter

weather and rainfall parameters, submit them to the backend, and view the flood prediction results in a clear and understandable format.

Activity 3.1: Design the User Interface

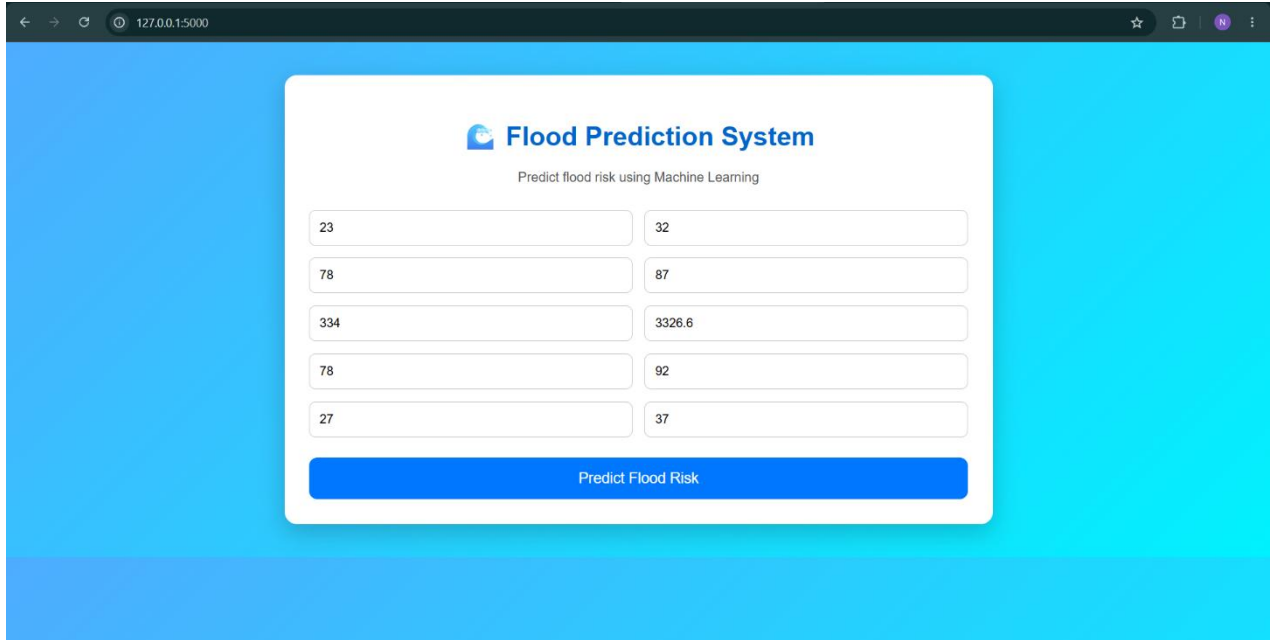
Develop an attractive and responsive user interface using **HTML, CSS, and JavaScript**. Create pages such as the Home page, Prediction page, About page, and Contact page to provide a seamless user experience.



The screenshot shows a web browser window displaying a web application titled "Flood Prediction System". The interface has a light blue background. At the top, there is a header with the title "Flood Prediction System" and a subtitle "Predict flood risk using Machine Learning". Below the header, there is a form with ten input fields arranged in two columns. The input fields are: Temperature (°C), Humidity (%), Cloud Cover (%), Annual Rainfall (mm), Jan-Feb Rainfall, Mar-May Rainfall, Jun-Sep Rainfall, Oct-Dec Rainfall, Average June Rainfall, and Sub Rainfall. At the bottom of the form, there is a large blue button labeled "Predict Flood Risk".

Activity 3.2: Develop Input Forms

Create interactive forms to collect the required weather and rainfall parameters, including Temperature, Humidity, Cloud Cover, Annual Rainfall, Jan–Feb Rainfall, Mar–May Rainfall, Jun–Sep Rainfall, Oct–Dec Rainfall, Average June Rainfall, and Subdivision Rainfall. Implement client-side validation to ensure that users enter valid input values.



Flood Prediction System

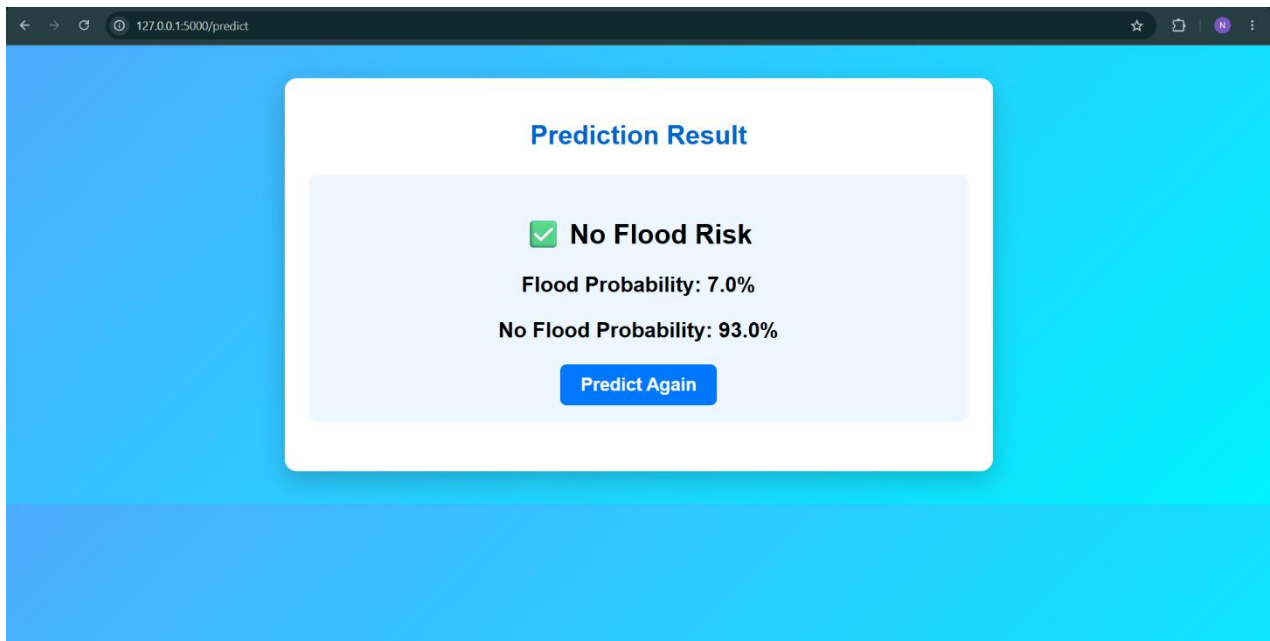
Predict flood risk using Machine Learning

23	32
78	87
334	3326.6
78	92
27	37

Predict Flood Risk

Activity 3.3: Display Prediction Results

Design the results page to present the Flood Risk Status (High/Low), Flood Probability (%), and Safety Recommendations in an easy-to-understand format. Use appropriate colors and layouts to improve readability and enhance the overall user experience.



Prediction Result

✓ **No Flood Risk**

Flood Probability: 7.0%

No Flood Probability: 93.0%

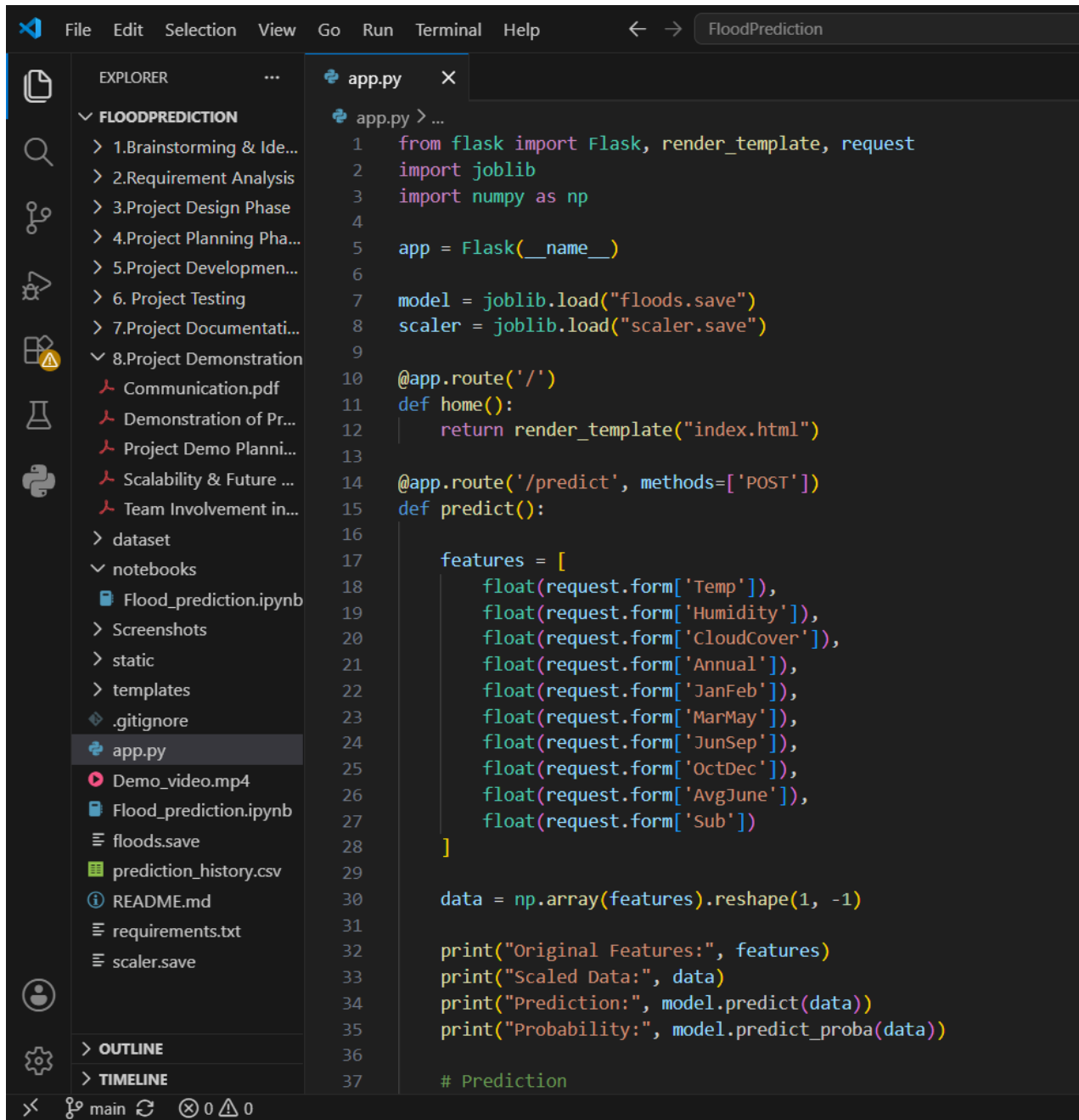
Predict Again

Milestone 4: Backend Development and System Integration

This milestone focuses on developing the backend of the Rising Waters – Flood Prediction System using the Flask framework. The backend is responsible for processing user inputs, validating the data, loading the trained machine learning model, and generating flood predictions. It also integrates the frontend with the machine learning model to provide real-time prediction results through the web application.

Activity 4.1: Develop the Flask Backend

Develop the backend using the Flask framework and implement the required routes for the Home page, Prediction page, About page, and Result page. Configure the application to receive user inputs from the frontend and process requests efficiently.



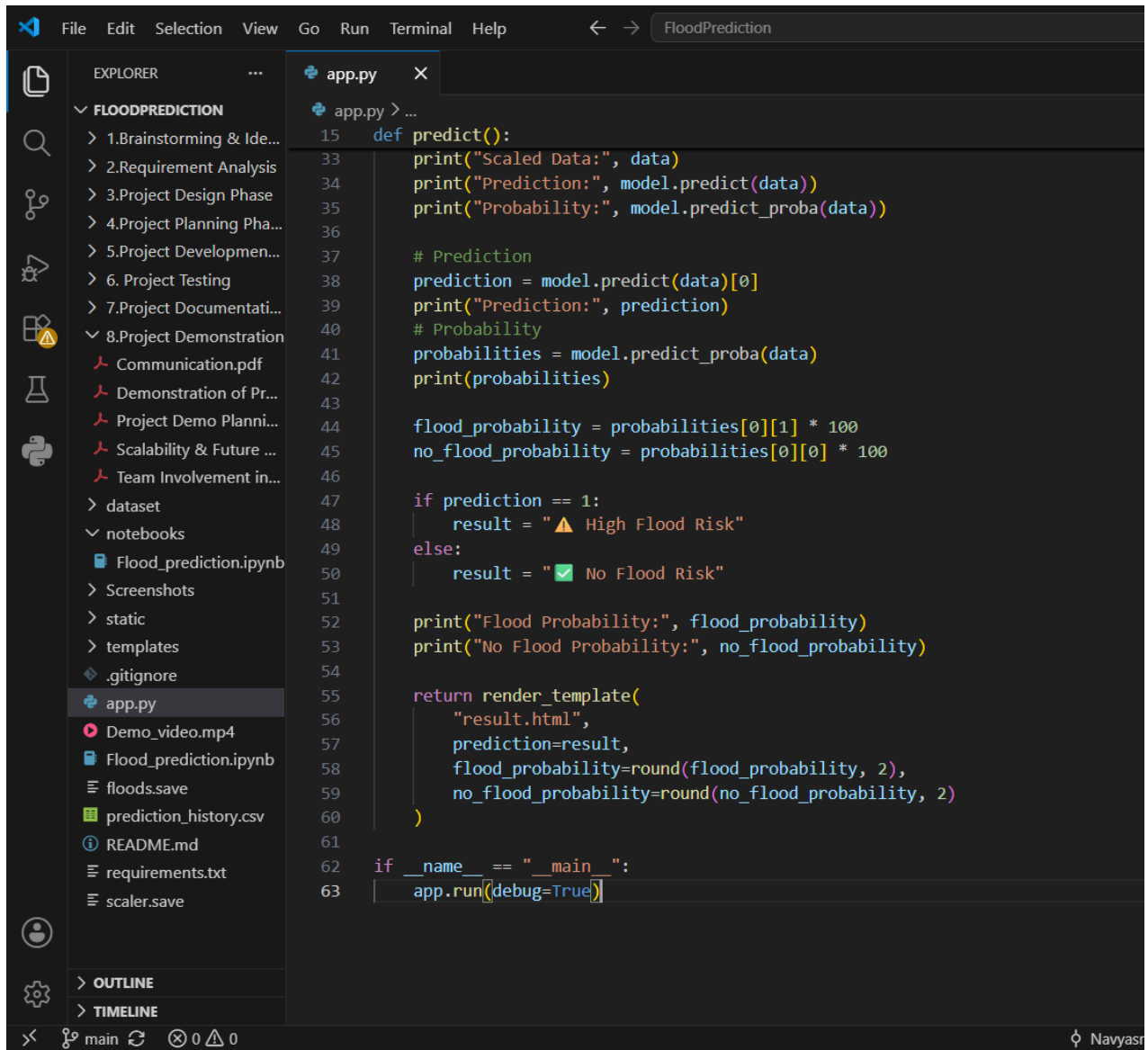
The screenshot shows a VS Code editor with a project named 'FLOODPREDICTION'. The Explorer sidebar on the left lists the project structure, including folders for 'dataset', 'notebooks', 'static', and 'templates', and files like 'app.py', 'Demo_video.mp4', 'Flood_prediction.ipynb', 'floods.save', 'prediction_history.csv', 'README.md', 'requirements.txt', and 'scaler.save'. The main editor window displays the code for 'app.py'.

```

1  from flask import Flask, render_template, request
2  import joblib
3  import numpy as np
4
5  app = Flask(__name__)
6
7  model = joblib.load("floods.save")
8  scaler = joblib.load("scaler.save")
9
10 @app.route('/')
11 def home():
12     return render_template("index.html")
13
14 @app.route('/predict', methods=['POST'])
15 def predict():
16
17     features = [
18         float(request.form['Temp']),
19         float(request.form['Humidity']),
20         float(request.form['CloudCover']),
21         float(request.form['Annual']),
22         float(request.form['JanFeb']),
23         float(request.form['MarMay']),
24         float(request.form['JunSep']),
25         float(request.form['OctDec']),
26         float(request.form['AvgJune']),
27         float(request.form['Sub'])
28     ]
29
30     data = np.array(features).reshape(1, -1)
31
32     print("Original Features:", features)
33     print("Scaled Data:", data)
34     print("Prediction:", model.predict(data))
35     print("Probability:", model.predict_proba(data))
36
37     # Prediction
  
```

Activity 4.2: Integrate the Machine Learning Model

Load the trained Random Forest model and the StandardScaler using Joblib. Validate and preprocess the user inputs before passing them to the machine learning model for prediction. Generate the flood risk status and probability based on the processed input data.



```

15 def predict():
33     print("Scaled Data:", data)
34     print("Prediction:", model.predict(data))
35     print("Probability:", model.predict_proba(data))
36
37     # Prediction
38     prediction = model.predict(data)[0]
39     print("Prediction:", prediction)
40     # Probability
41     probabilities = model.predict_proba(data)
42     print(probabilities)
43
44     flood_probability = probabilities[0][1] * 100
45     no_flood_probability = probabilities[0][0] * 100
46
47     if prediction == 1:
48         result = "⚠️ High Flood Risk"
49     else:
50         result = "✅ No Flood Risk"
51
52     print("Flood Probability:", flood_probability)
53     print("No Flood Probability:", no_flood_probability)
54
55     return render_template(
56         "result.html",
57         prediction=result,
58         flood_probability=round(flood_probability, 2),
59         no_flood_probability=round(no_flood_probability, 2)
60     )
61
62 if __name__ == "__main__":
63     app.run(debug=True)
  
```

Activity 4.3: Connect Frontend and Backend

Integrate the frontend with the Flask backend to enable seamless communication between the user interface and the prediction model. Display the prediction results, including **Flood Risk (High/Low)**, **Flood Probability (%)**, and **Safety Recommendations**, on the results page.

This milestone ensures that the web application functions as a complete end-to-end system by connecting the user interface with the trained machine learning model and delivering accurate flood predictions in real time.

Milestone 5: Testing, Deployment, and Documentation

This milestone focuses on verifying the functionality of the Rising Waters – Flood Prediction System, deploying the application, and preparing the final project documentation. The application is tested with different weather and rainfall inputs to ensure accurate flood predictions. After successful testing, the project is organized with proper documentation and uploaded to GitHub for version control and submission.

Activity 5.1: System Testing and Validation

Perform comprehensive testing of the application using different weather and rainfall data. Verify that the system correctly predicts High Flood Risk or Low Flood Risk, displays the prediction probability, and handles invalid user inputs effectively.

Activity 5.2: Deployment and Version Control

Run the Flask application in the local development environment and ensure all modules function correctly. Upload the complete project, including source code, documentation, and required files, to GitHub for version control and project management.

Activity 5.3: Documentation and Project Demonstration

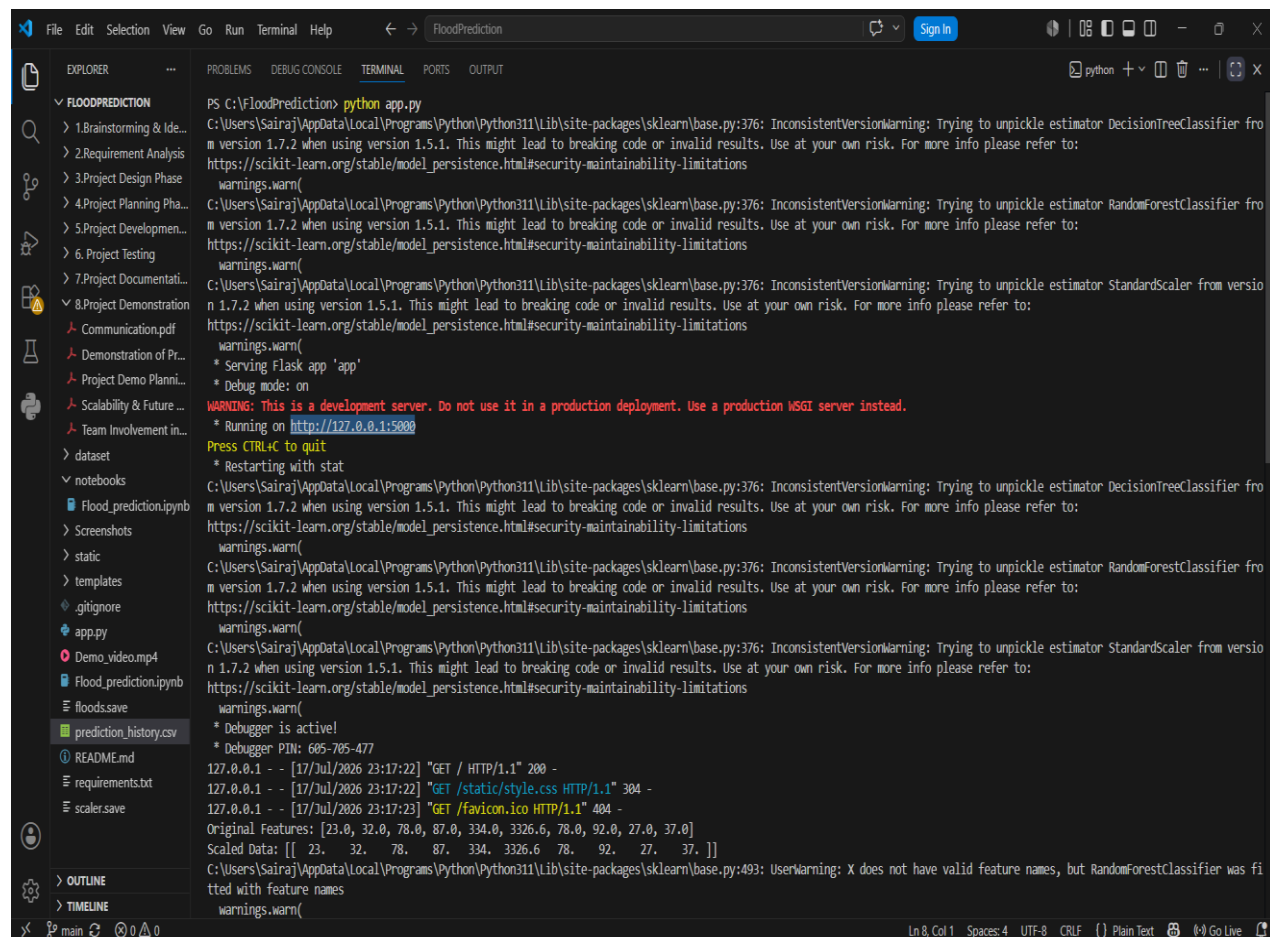
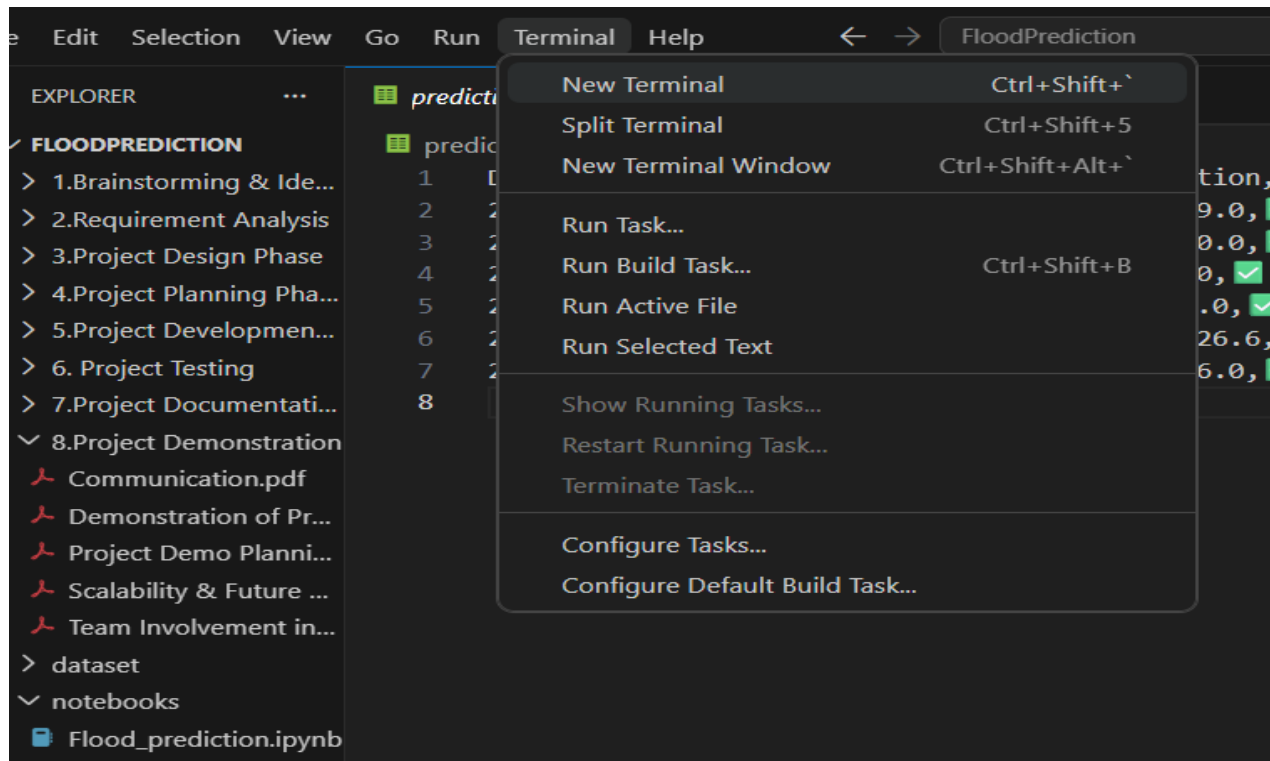
Prepare the project documentation, including the system architecture, workflow, implementation details, screenshots, and testing results. Create a demonstration video showcasing the application's features and working process for presentation and final project submission.

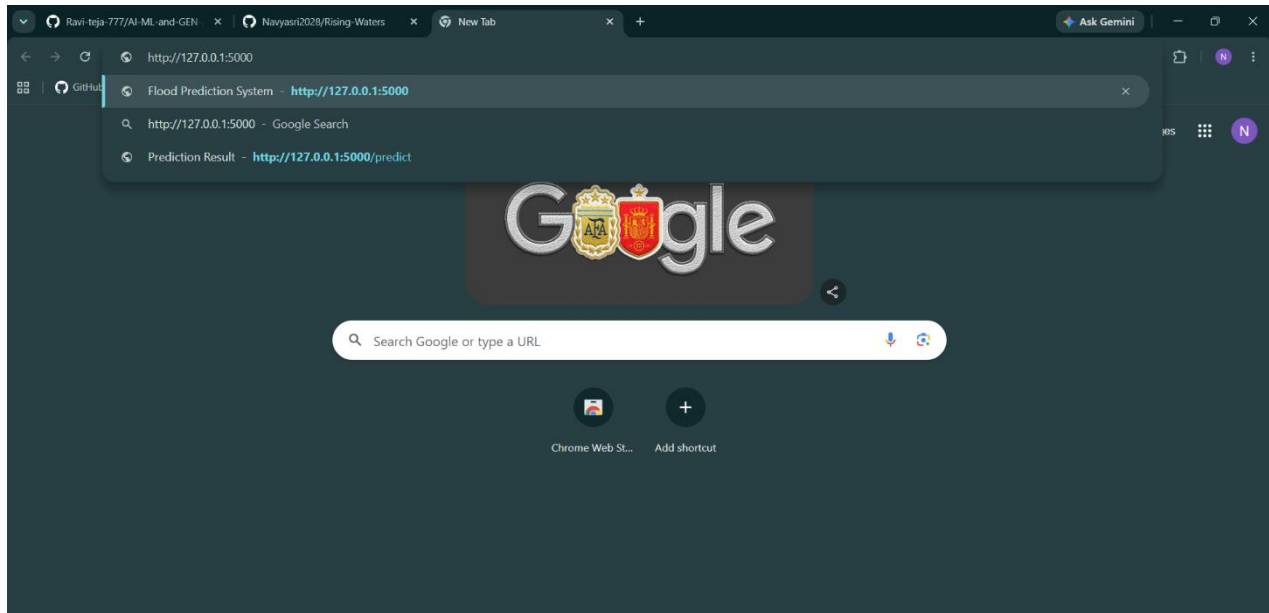
This milestone ensures that the **Rising Waters – Flood Prediction System** is fully tested, properly documented, and ready for deployment and evaluation.

Exploring the Website's Web Pages:

First, Open Project Document “FloodPrediction” in VS Code and then click on Terminal>New Terminal

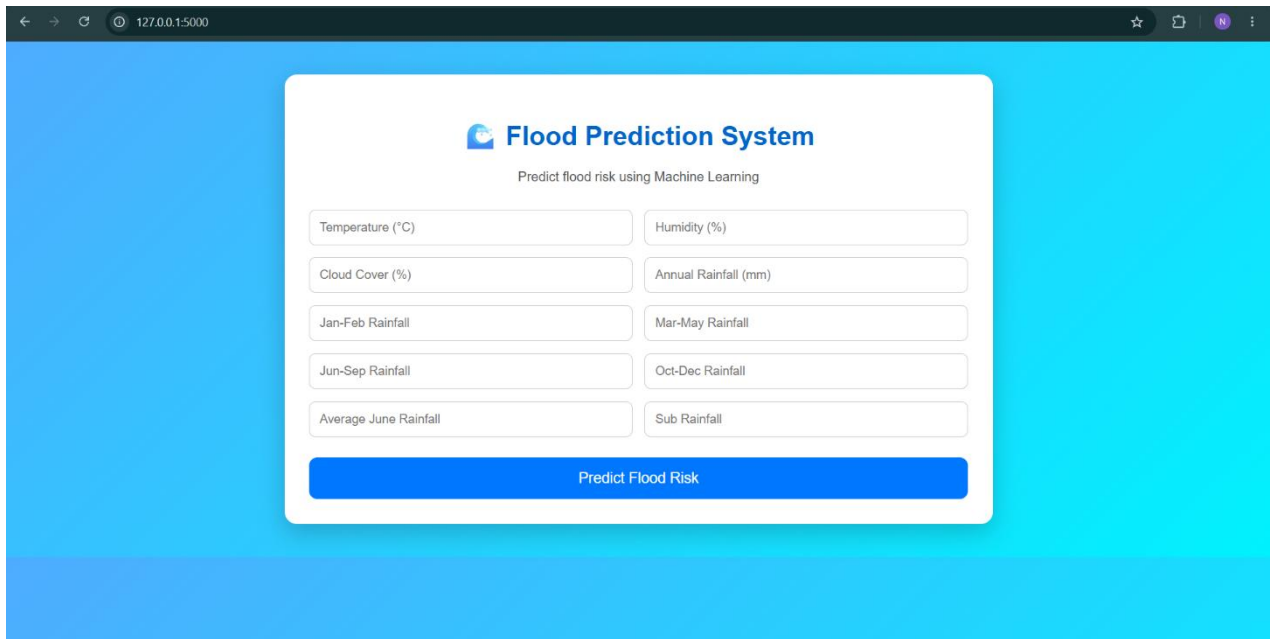
And then type “python app.py” and click Enter and it execute and one link will appear among them that is <http://127.0.0.1:5000> and copy this and paste in browser then it opens one UI page that asks to enter all weather related questions and predict...





Paste the link in above web browser and then click Enter. It appears like below:

Home/Generator Page:



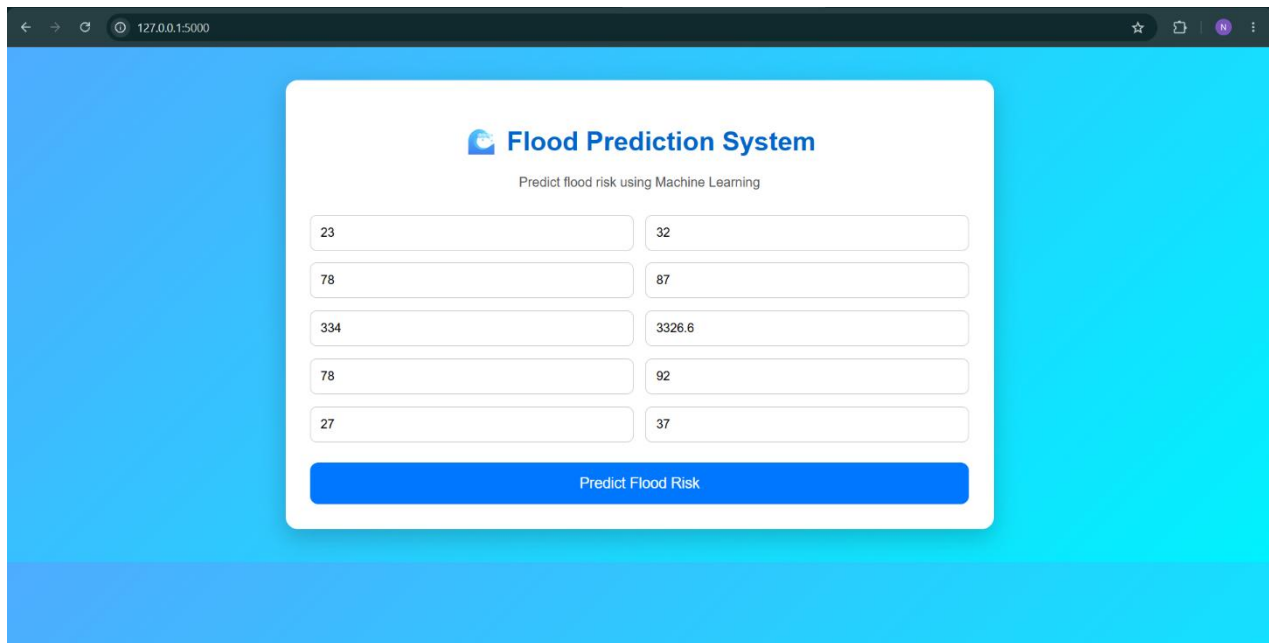
Flood Prediction System
Predict flood risk using Machine Learning

Temperature (°C)	Humidity (%)
Cloud Cover (%)	Annual Rainfall (mm)
Jan-Feb Rainfall	Mar-May Rainfall
Jun-Sep Rainfall	Oct-Dec Rainfall
Average June Rainfall	Sub Rainfall

Predict Flood Risk

Description: The above page is front-end page for Predicting Flood Prediction System. It asks to enter values related to floods and weather and then click predict button.

Input Section:



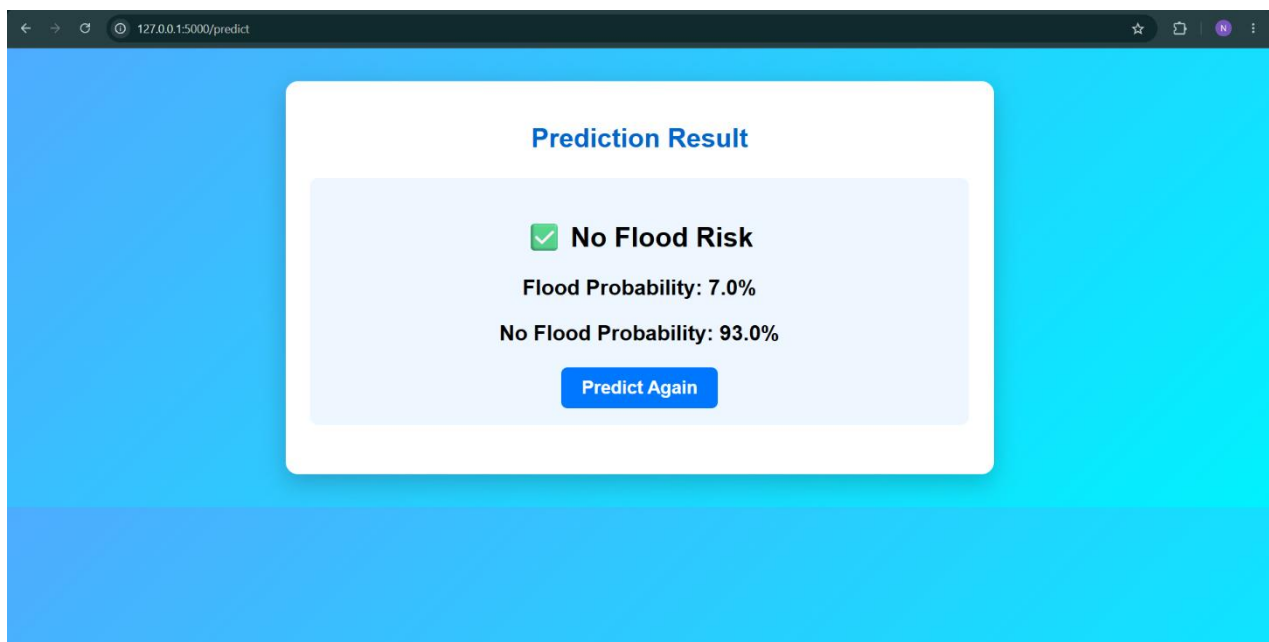
Flood Prediction System
Predict flood risk using Machine Learning

23	32
78	87
334	3326.6
78	92
27	37

Predict Flood Risk

Description: It takes some inputs related to weather for Predict the Floods.

Results Section:



Prediction Result

✓ **No Flood Risk**

Flood Probability: 7.0%

No Flood Probability: 93.0%

Predict Again

Description: Above page is Result Page that is Prediction Result Page. Based on inputs we provided and dataset we given, the model trained and predict the result as whether there is Flood Risk or not and it will also give Flood Probability and No Flood Probability. And it also there "Predict Again" Button.

Conclusion:

The Rising Waters – A Machine Learning Approach to Flood Prediction project successfully demonstrates the application of machine learning in predicting flood risk using historical weather and rainfall data. By utilizing the Random Forest Classifier, the system analyzes key environmental parameters such as temperature, humidity, cloud cover, and seasonal rainfall to provide accurate flood risk predictions along with the corresponding probability.

The integration of the trained machine learning model with a Flask-based web application enables users to enter weather information through a simple and user-friendly interface and receive real-time prediction results. This system can support government authorities, disaster management agencies, farmers, and local communities in taking timely preventive measures to reduce the impact of floods.

Overall, the project provides an efficient, reliable, and scalable solution for flood prediction. With future enhancements such as real-time weather data integration, GIS-based flood mapping, and automated alert systems, the Rising Waters application has the potential to become a more comprehensive decision-support system for disaster preparedness and flood risk management.